Parul® University

Software Engineering (303105253)

Prof. Divyesh Hariyani
Assistant Professor
CSE – Cyber Security

Pacel University

- DIGITAL LEARNING CONTENT

Unit-6

Software Testing and Quality Assurance

Parul® University

DIGITAL LEARNING CONTENT

Contents- Software Testing

- Software Testing Concepts
- Psychology of testing
- Levels of testing
- Testing Process-
    - test plan, test case design, Execution
- Black-Box testing
    - Boundary value analysis
- Pair wise testing
- state based testing,
- White-Box testing
    - criteria and test case generation and tool support

Image source : Google

Parul'
University

DIGITAL L£ARNING CONTENT

Contents- Quality Assurance

- Quality Control
- Assurance, Cost, Reviews
- Sokware Quality Assurance
- Approaches to SQA
- Reliability
- Quality Standards- ISO900O And 9001

Parul"
University

DIGITAL LEARNING CONTENT

Concepts

- Testing is the process of exercising a program with the specific intent of finding errors prior to delivery to the end user.
- To understandtestingtechniquesthat are geared to discover program faults
- To introduce guidelines for interface testing
- To understand specific apprDaches to object- oriented testin8
- To understand the principles of CASE tool support for testing

Psychology of testing

- All tests should be traceable to customer requirements.
- Tests should be planned long before testing begins.
- The Pareto principle applies to sokware testing.
- Testing should begin "in the small" and progress toward testing "in the large."
- Exhaustive testing is not possible.
- To be most effective, testing should be conducted by an independent third party.

Software testing phases

- **Component testing**
  - Individual program components testing
  - Tests are derived from the developer's experience
- **Integration testing**
  - Testing of groups of components integrated to create a system or sub-system
  - Tests are based on a system specification

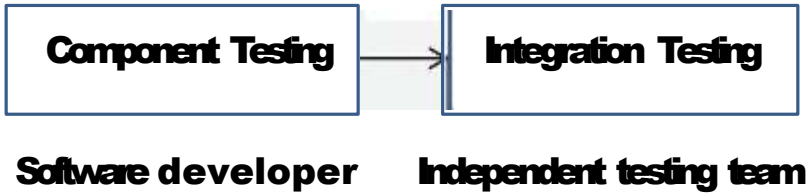


Figure 6.1 Software testing phase

Testing process

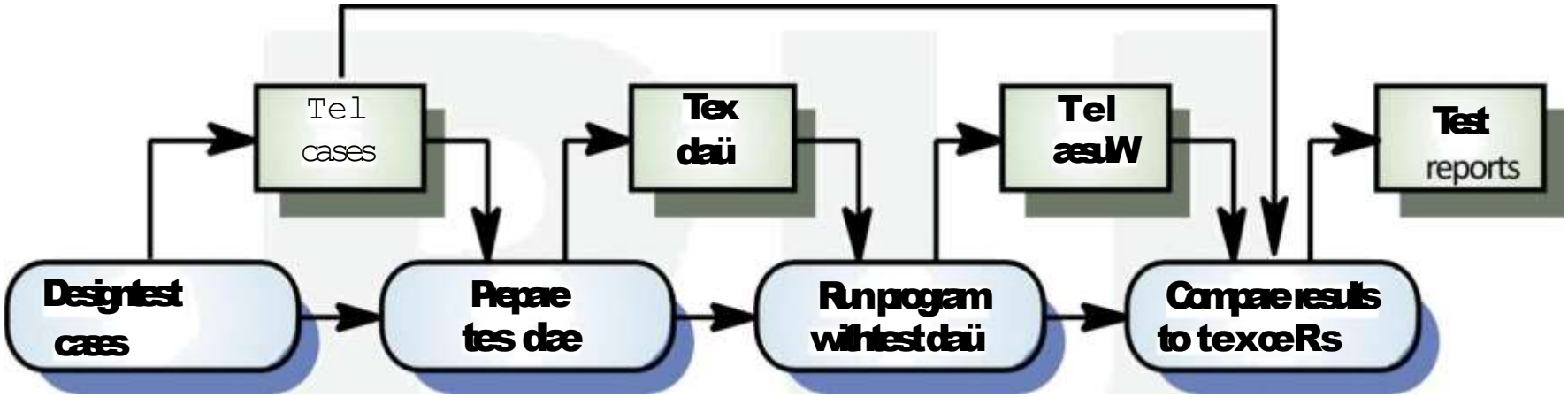


Figure 6.2 Software testing process

Test plan

- Test planning, the most important activity to ensure that there is initially a list of tasks and milestones in a baseline plan to track the progress of the project.
- It also defines the size of the test ePort.
- It is tËe main document often called as master test plan or a project test plan and usually developed during the early phase of the project.

Paml°
University

- DIGITALLEARNINGCONTENT

Test Planning Activities

- To determine the scope and the risks that need to be tested and that are NOT to be tested.
- Documenting Test Strategy.
- Deciding Entry and Exit crïterîa.
- Evaluating the test estimate.
- Planning when and how to test and deciding how the test results will be evaluated, and defining test exit criterion.

En ”ng that the test documentation generatoe repeatable test s

Test case design

- Test case design methods provide a mechanism that can help to ensure the completeness of tests
- Provide the highest likelihood for uncovering errors in software.

Any product or system can be tested on two ways:

1. Knowing the specified function that a product has been designed to perform; (Black Box)
2. knowing the internal workings of a product, tests can be conducted to ensure that "all gears mesh," that is, internal operations are performed according to specifications (White box testing)

Pam%
University

Test Execution

- Test execution is the process Df exesuting the code and comparing *the* expected and aclual *results.*

Fc›llowing factors arm to be cc›nsi6ared for a test eXeCution process:

- Based Dn a risk, select a subset Df test suite lc› be exec used for this cycle.
- Assign the test cases in each lest suite to testers fc›r executic›n.
- Execute tests, repDrl bugs, and caplum test status continuously.
- Resolve blDcking issues as they arise.
- RepDrt Status, adjust assignmenfs, and reconsider plans and priDrilies daily.
- Repos test cycle findings and status.

Levels of testing

- There are different levels during the process of testing.
- Levels of testing includE' different methodologies that can be used while conducting sDftware testing.

Tha main lavals nf soHwara tasting ara -

- FcnotionaiTesting
- NDn-functional Testing

Functional Testing

* This is a type of black-box testing that is based Dn the specifications of the sofhvare that is to be tested.

• Functional testing of a sofhvare is conducted on a complete, integrated system to evaluate the system's compliance \vith its specified requirements.

There are five xepsth# ae invobedwhi&. teuing an applicatÔnfor functionality.

1. The determination of the functionality that the intended application is meant io perform.
2. The creation of test data based on the specifications of the application.
3. The output based on the test data and the specificÄOFIS Dfthe application.
4. The writing of test scenarîos and the execufion oJtest cases.
5. The comparison of actual and expected results based on the executed test cases.

Unit Testing

- This type of testing is performed by developers before the setup is handed over to the testing team to formally execute the test cases.
- Unit testing is performed by the respective developers on the individual units of source code assigned areas.
- The developers use test data that is different from the test data of the quality assurance team.
- The goal of unit testing is to isolate each part of the program and show that individual parts are correct in terms of requirements and functionality.

Integration Testing

- IntegratiDn testing is defined as the testing of combined parb Df an application to determine if they function correctly.
- IntegratiDn testing can be dDne in twD Wd : BOttom-up integratiDD testing and Top-dDwn integraéon testing.

1. BoUom-up integration

This testing begins with unit testing, followed by tests of progressively higher-level combinations of units called modules or builds.

2. Top-down integraion

In this testing, the highest-level modules are tested first and progressively, lower-level modules are tested thereaker.

System Testing

- System testing tests the system as a whole.
- This type of testing is performed by a specialized testing team.

System tes ing is important because of the following reasons-

- System testing is the first step in the Software DevelDpment Life Cycle, where the application is tested as a whole.
- The application is tested thoroughly tD verify that it meets the funEtional and technical specifications.
- The application is tested in an environment that is very close tD the production environment where the application will be deployed.
- System testing enables us to test, verify, and validate both the business requirements as well as the application architecture.

Non-Functional Testing

- Non-functional testing involves testing a softwa re from the requirements which are nonfunctiDnal in nature but impDrtant such as performance, security, user interface, etc.

.Some of the importarn and commonly used non-functi'onaitesting types are discussed below.

- Performance Testing
- 1oad Testing
- Stress Testing
- **Usability Testing**
- **Security Testing**
- **Portability Testing**

Black-Box testing

- An approach to testing where the program is considered as a 'black-box'
- The program test cases are based on the system specification
- Test planning can begin early in the software process

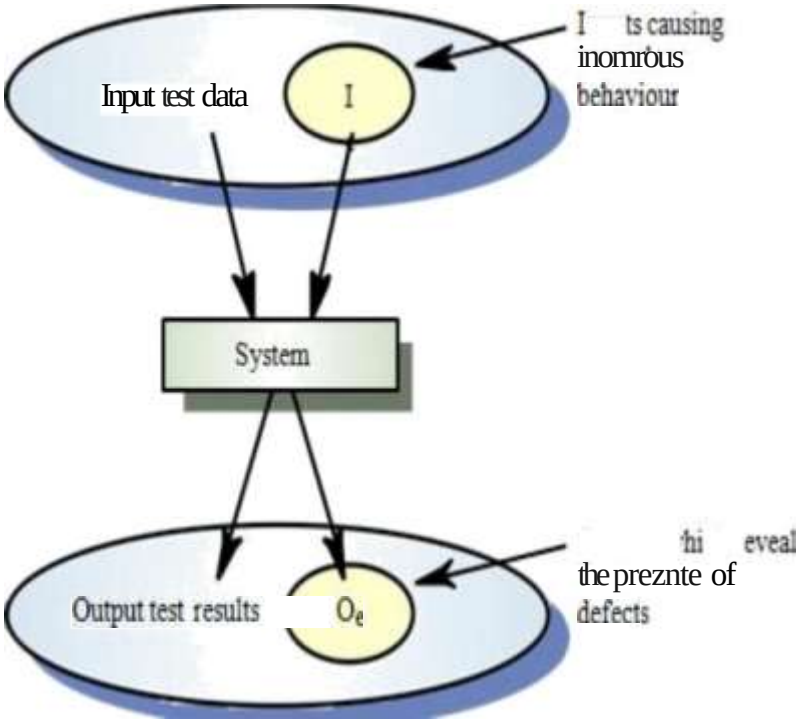


Figure 6.3 Black box testing

Continue…

- Also called *behavioraltesting,* focuses on the functional requirements of the software.
- It enables the software engineer to derive sets of input conditions that will fully exercise all functional requirements for a program.
- Black-box testing is not an alternative to white-boxtechniques but it is complementary approach.

- Black-boxtesting ahem@s to find errors in the following caegories:
  - Incorrect or missing functions,
  - Jnterface errors,
  - Errors in data structures or external data base access.
  - Behavior or performance errors,
  - Initialization and termination errors.

Continue…

- Black-box testing purposely ignored control structure, attention is fOCUSEd on the information dDmain.
- Tests are designed to answer the fDllowing questions:
  - How is functional validity tested?
  - HDw is system behaviDr and perfDrmance tested?
  - What classes of input will make gDod test cases?
- By applying black-bDx techniques, we derive a set Df test cases that satisfy the following criteria
  - Test cases that reduce the number of additional test cases that must be designed to achieve reasonable testing (|,e minimize effort and time)
  - Test cases that tell us something about the presence or absence of classes of errors

Guidelines for BVA

- If an input condition specifies a range bounded by values a and *b*, test cases should be designed with values *a* and *b* and just above and just below a and d.
- If an input condition specifies a number of values, test cases should be developed that exercise the minimum and maximum numbers. Values just above and below minimum and maximum are also tested.
- Apply guidelines 1 and 2 to output conditions.
- If internal program data structures have prescribed boundaries be certain to design a test case to exercise the data structure at its boundary

Pair wise testing

- Pair-wise Testing is a type of software testing in which permutation and combination methDd is used to test the software.
- used to test all the possible discrete combinations of the parameters involved.
- Ali is a P&C based method, in which to test a system Dr an application, for each pair of input parameters of a system, all possible discrete combinations of the parameters are tested.
- ay using the cDnventional or exhaustive testing approach it may be hard to test the system
- but by using the permutation and combination method it can be easily done.

Advantages of Pair-wise Testing

The advantages of pair-wise testing are:

- Pair-wise testing reduces the number of execution of test cases.
- Pair-wise testing increases the test coverage almost upto hundred percentage.
- Pair-wise testing increases the defect detection ratio.
- Pair-wise testing takes less time to complete the execution of the test suite.
- Pair-wise testing reduces the overall testing budget for a project.

Disadvantages of Pair-wise Testing

The disadvanQges of pair-wise testing are:

- Pair-wise testing is not beneficial if the values Df the variables are inappropriate.
- In pair-wise testing it is possible to miss the highly probable combination while selecting the test data.
- In pair-wise testing, defect yield ratiD may be reduced if a combination is missed.
- Pair-wise testing is not useful if combinations of variables are not understood correctly.

State based testing

- To understand the objects that are modeled in software and the relationships that connect these Dbjects.
- Next step is ID define a series of tests that verify "all objects have the expected relationship to one another.
- Stated in other way:
  - Create a graph Df Important objects and their relatiDFIShips
  - Develop a series of tests that will cover the graph, So that each Dbject and relationship is exercised and errors are uncovered.

Begin by creating graph -

- a collection of nodes that represent objects
- links that represent the relationships between Dbjects
- node weights that describe the properties of a nDde
- link weights that describe some characteristic of a link.

Continue…

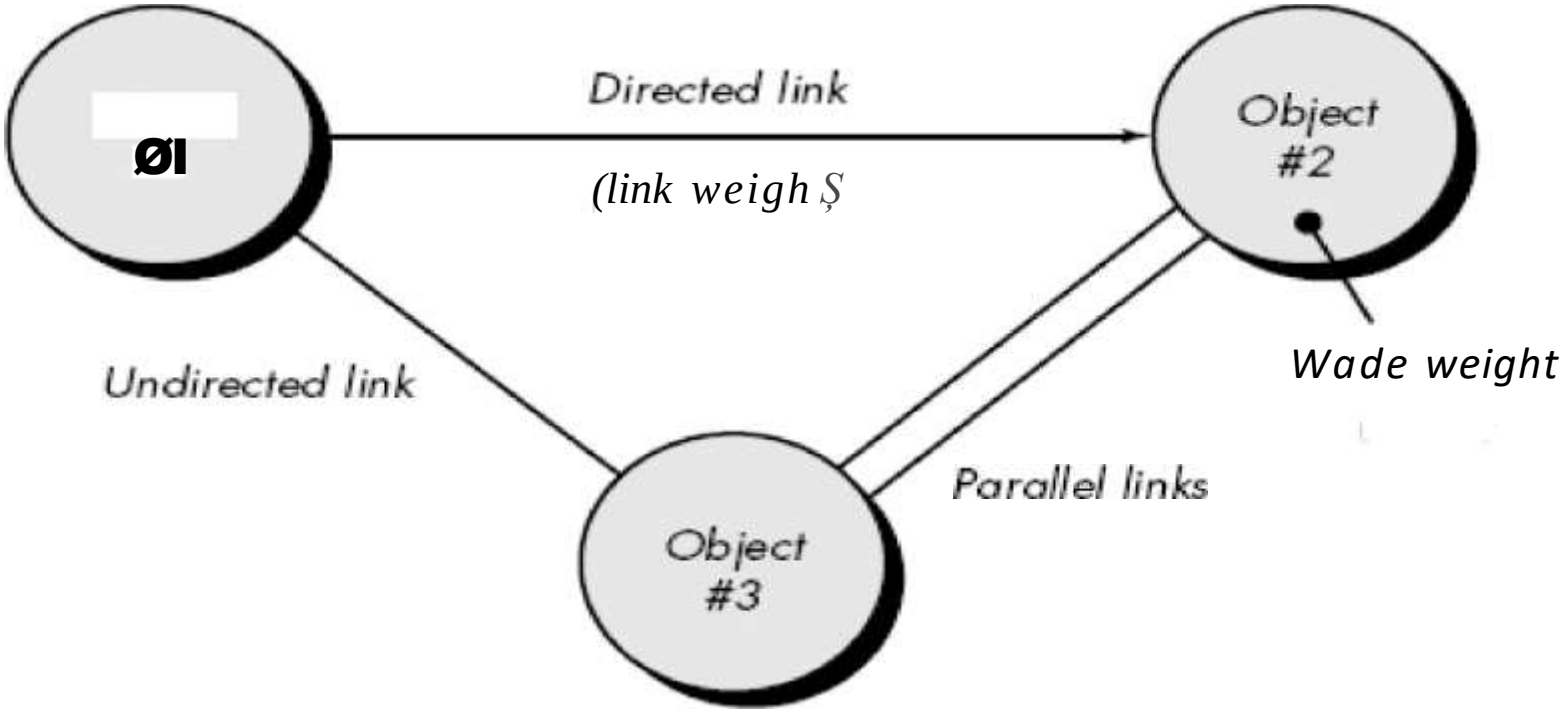


Figure 6.4 Graph notation

Continue…

- Nodes are represented as circles connected by links that take a number of different forms.
- A directed link (represented by an arrow) indicates that a relationship moves in only one direction.
- A bidirectional link, also called a symmetric link, implies that the relationship applies in both directions.
- Parallel links *are* used when a number of different relationships are established between graph nodes.

Continue…

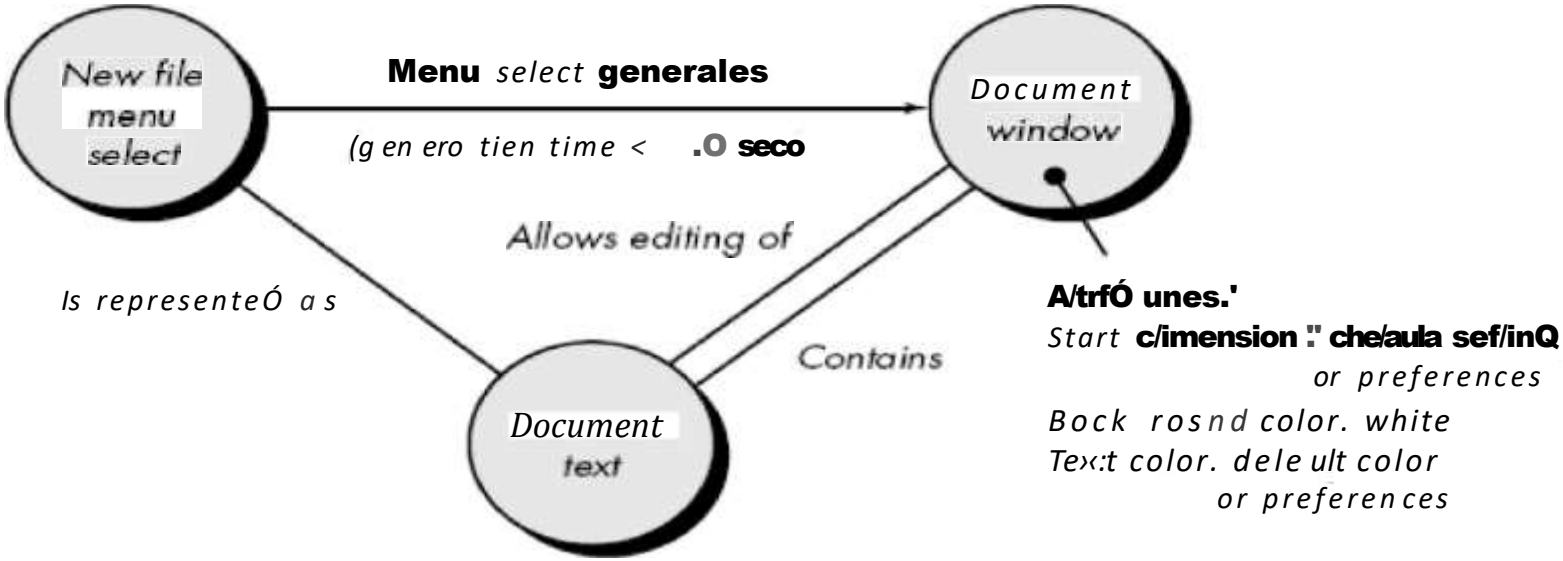


Figure 6.5 Simple example

Continue…

- Object d1 = new file menu selaEt
- Object #2 = document window
- Object d3 = document text

Referring to example figure, a menu select on naw fila generates a documant window.

- The link weight indicates that the window must be generated in less than 1.0 second.
- The node weight Df documem window provides a list of the window attributes that are to be expected when the window is generated.
- An undirected link establishes a symmetric relationship between the new file menu select and document text,

parallel links indicate relationships between documem window and document text

Continue…

- Number of behavioral testing methods that can make use of graphs:

Transaction flow modeling.

- The nodes represent steps in some transactiDn and the links represent the logical connection between steps

Finite state modeling.

- The nodes represent different user observable states of the software and the links represent the transitions that occur to r«ove from state to state. (Starting point and ending point)

Data flow modeling.

- The nodes are data objects and the links are the transformations that Dccurto translate Dne data object into another.

Timing modaling.

- The nodes are program objects and the links are the sequential connections between those objects.
- xLink weights are used to specify the required execution times as the program executes.

White-Box testing

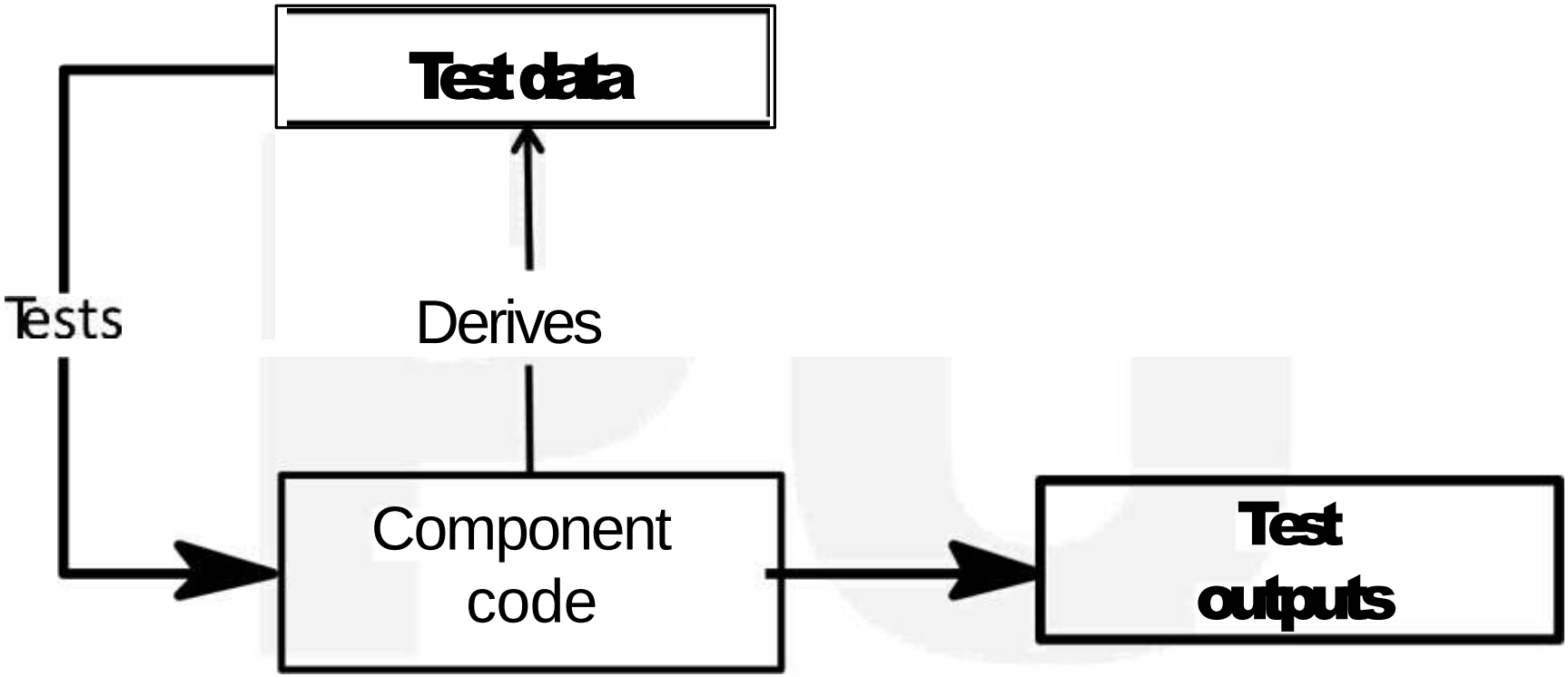


Figure 6.6 Architecture of White -Box testing

Continue…

- White-box testing is a test case design rTiethod
- Uses the control structure of the procedural design to derive test cases
- 'Using white-box testing methods, the sDftware engineer can derive test cases that
- Guarantee that all independent paths within a module have been exercised at least once
- Exercise all logical decisions on their true and false sides
- Execute all loops at their boundaries and within their operational bounds
- Exercise internal data structures to ensure their validity.

Basis path testing

Basis path testing is a white-box testing technique

- To derive a logical complexity measure of a procedural design.
- Test cases derived to exercise the basis set are guaranteed to execute every statement in the program at least one time.

Methods:

1. Flow graph notation
2. Independent program paths or Cyclai¿,pti¿c complexity
3. Deriving test cases
4. Graph Matrices

Flow Graph Notation

- **Start with simple notation for the representation of control flow (called flow graph). It represent logical control flow.**

The structured constructs in flow proph form:

Sequence

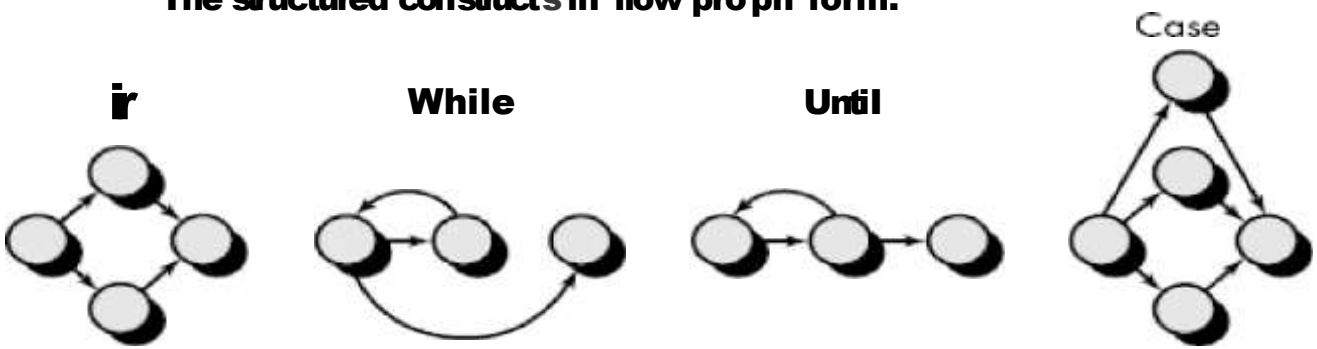


Figure 6.7 Flow graph notation

Continue…

- A sequence of process boxes and decision diamond can map into a single node.
- The arrows on the flow graph, called edges or links, represent flow of control and are parallel to flowchart arrows.
- An edge must terminate at a node, even if the node does not represent any procedural statement.
- Areas bounded by edges and nodes are called regions.
- When compound condition are encountered in procedural design, flow graph becomes slightly more complicated.

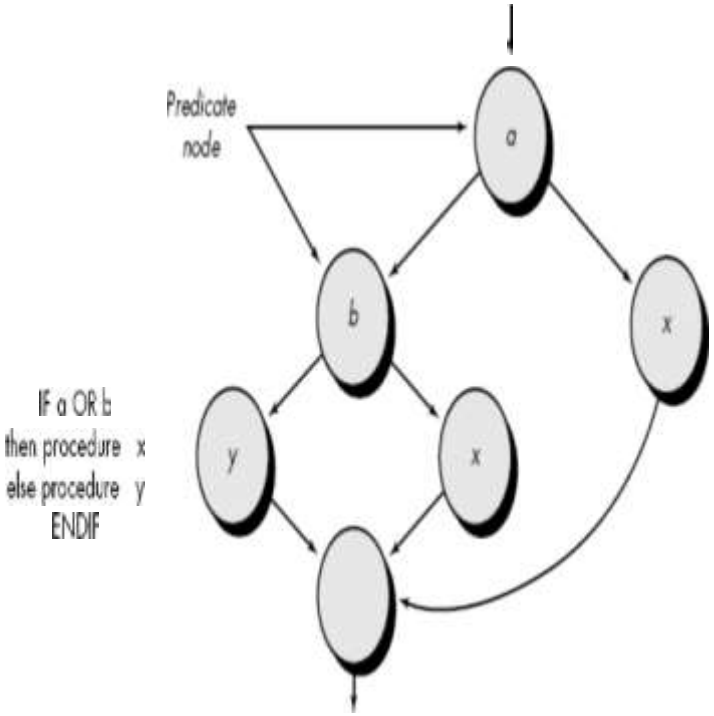


Figure 6.8 Compound logic

Continue…

When we translating PDL segment into flow graph, separate node is created for each condition.

Each node that contains a condition is called predicate node and is characterized by two or more edges comes from it.

Independent program paths of Cyclomatic complexity

- An independent path is any path through the program that introduces at least Dne new set of prDcessing statement or new ConditIDU.

For example, a set of independent paths for flow graph:

Path 1: 1-11
Path 2: 1-2-3-4-5-1O-1-11
Path 3: 1-2-3-6-8-9-1-11
Path 4: 1-2-3-6-7-9-1-11

- NDte that each new path introduces a new edge.
  - The path 1-2-3-4-5-10-1-2-3-&8-9-1-11 is not considered to an independent path because it is simply a combination of already specified paths and does not traverse any new edges.
  - Test cases should be designed to force execution of these paths (basis set).

Continue…

- Tyc,lqttsttis complexity is a software metrics that provides a quantitative measure of the logical complexity ofa program.

It defines no. of independent paths in the basis set and also prDvides number of test that must be conducted.

One of three ways to compute cystitstTrafic complexity:

1. The no. o/ *regions* corresponds to the cycloiTj,atc, complexity.
2. SsSJTmpgc complexity, V(G), for a flow graph, G, is defined as
   V(G) E- N+ 2
   where E is the number of flow graph edges, N is the number offlow graph nodes.
3. ,CsCtsittitic CDmplexity, V(G), for a flow graph, G, is also defined as
   V(G) = P+ 1
   where P is the number Df predicate nodes edges

So the value ofV(G) provides us with upper bound oftest cases.

Deriving Test Cases

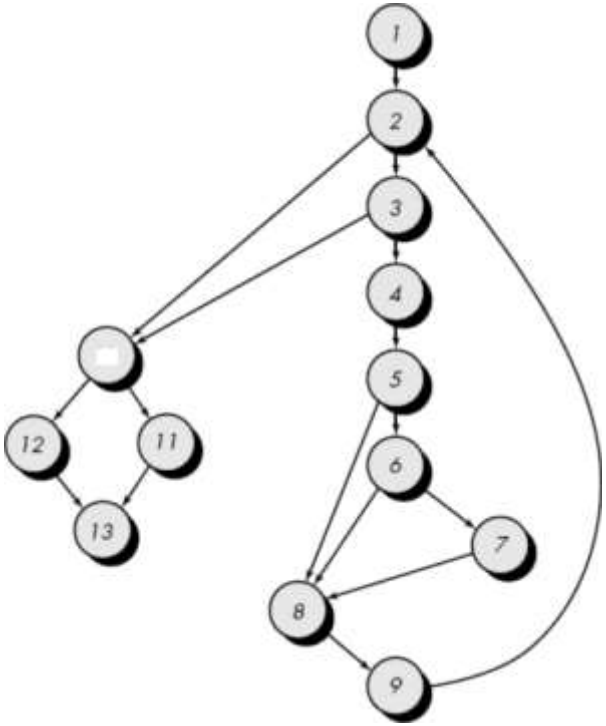
- It is a series of steps method.
- The procedure average depicted in PDL.
- Average, an extremelysimple algorithm, contains compound CDNdltiDns and loops.

To derive basis set, follow the steps.

1. Using the design or code as a foundation, draw a corresponding flow graph.
   - A flow graph is created by numbering those PDL statements that will be mapped into corresponding flDw graph nodes.

Pam%
University

Deriving Test Cases



Flow graph for the procedure average

Continue…

2. Determine the lom }c complexrty of the resuhant flow graph.
   - V(C) can be determined •/ithout developing aflo vgraph by counting all conditional statements in the PDL (for the procedure average, compound conditions count as Evo) and adding 1
   - *fiG)* – 6 regions
   - V(C) = 17 edges - 13 nodes+ 2 = 6
   - *fiG)* – 5 predicate no:des+ 1 = 6
3. Determine a basis set of linearly independent paths
   - The value of U{C) provides the number of lin early ind€?pendcznt paths through the program control structure.
   - path 1. 1-2-10-11-13
   - path 2. 1-2-10-12-13
   - path 3. 1-2-3-10-11-13
   - path 4: 1-2-3-4-5-B-9-2-. .
   - path 5. 1-2-5-4-5-6-8-9-2-...
   - path 6. 1-2-3-4-5-6-7-B-9-2-...
   - The ellipsis (. .) following paths 4, 5. and 6 indicates that any path through the remainder of the control structure is acceptable.

Continue…

4. Prepare test cases that will force execution of each path in the basis set.
   - Data should be chosen so that conditiDns at the predicate nodes are appropriately set as each path is tested.
   - .Each test case is executed and compared ID expected results.
   - Once all test cases have been completed, the tester can be sure that all statements in the program have been executed at least once.

Graph Matrices

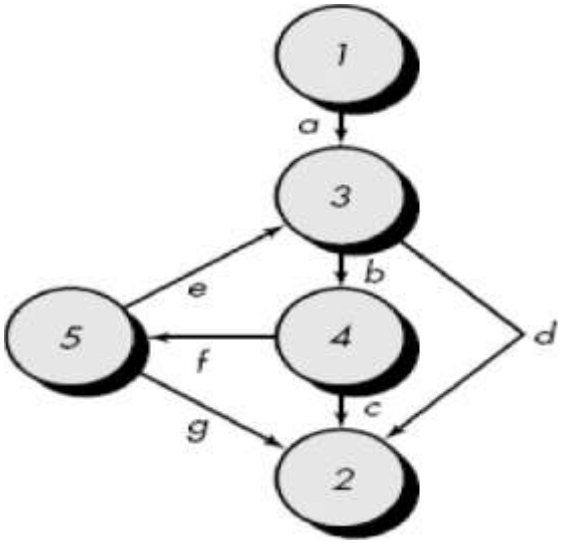
A graph matrix is a square matrix ‹whose size (i.e., number of FDʌs and columns) is equal to the number of nodes on the flow graph.

- Each roʌv and column. CDrresponds to an identified node, and matrix entries correspond to connections (an edge) beLveen rDoles.
- Each node Dn the floʌ graph is identify by numbers, ›ʌhile each edge is identify by letters.
- The graph matrix is nothing more than a tabular representation of a flov gfaph.
- By adding a link›veighttsach matrix entry, the graph matrix can become a powerful tool fof evaluating pfDgFaʌi control structure during testing.

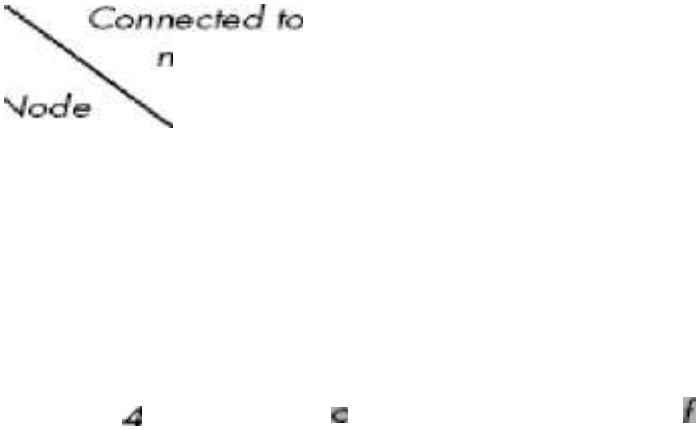
The link‹veight provides additional information aboul control floʌ. In its simplest form, the link weight is 1 (a connection exists) or 0 (a connection does not exist).

Pam%
University

Continue…



Flow graph



Graph matrix

Connection matrix

Node \ Connected node	1

Figure 6.9 Connection matrix

Connection matrix

- Each letter has been replaced with a 1, indicating that a connection exists (this graph maVix is called a *connection matrix).*
- In fig.(conneZion matrix) each row with two or more entries represents a predicate node
- We can dire4ly measure c;jcly¿t;a¿ic complexity value by performing arithmetic operations

CDnnections = Each rDW Total nD. of entries —1

V(G)= Sum of all connections + 1

Quality Control - Quality

- Quality as "a characteristic or attribute of something."
- Two kinds Df quality may be encountered:
  - Quality of design of a produCt inEreases, if the prDduct is manufactured according to specifications.
  - Quality of conformance is the degree to which the design specifications are fDllowed during manufacturing.

- In software developmern
  - Quality of design encompasses requirements, specifications, and the design of the system.
  - Quality of conformance is an issue focused primarily on implementation.
- User satisfaction = compliant product + good quality + delivery within budget and schedule

Quality control

- Quality control invokes the series of inspections, reviews, and tess used throughout the software process.
- Quality control includes a feedback loop to the process.
- A key concept of quality control is that all work products have defined, measurable specifications to which we may compare the output of each process.
- The feedback loop is essential to minimize the defects produced.

Quality Assurance

- Quality assurance consists of the auditing and reporting functions of management.
- If the data prDvided through quality assurance identify prDblem
- it is management's responsibilityID address the problems and
- apply the necessary resources to resolve quality issues.

Cost of Quality

- The cost of quality includes ah costs incurred in the pursuit of quality or in performing quality-related activities.

- Quality costs may be divided 3 mode of cost:
  - Prevention
  - Appraisa.l
  - Failure.

Pam%
University

Continue...

- Prevention costs include:
  - Quality planning
  - Formal technical revie\'vs
  - Testequipment
  - Training

- Appraisal costs include activities to gain insight into product
  - In-process and Inter-process inspection
  - Equipment calibration and maintenance
  - Testing

Pam%
University

Continue...

- Failure costs
  - Internal Failure Cost
    - rework
    - repair
    - failure mode analysis
- External Failure Cost
  - complaint resolution
  - product return and replacement
  - help line support
  - warranty work

Software Quality Assurance

Definition:

Conformance tD explicitly stated functiDnal and performance requirements, explicitly documented develDpment standards, and implicit characteristics that are expected of all professionally developed software.

- **Dafinition serves to amphasize thrae important points:**
  - **SDftware requirements are the foundation from which quality is measured.**
  - **Jf specified standards criteria are not follDwed, lack of quality will almost surely result.**
  - **A set of implicit requirements often goes unmentioned.**

SQA group Activity

- SQA group ma& up of software engineers, project managers, customers, sales peoge, and the individuals members.
- SQA group that has responsi@ily for quality assurance ganning, oversight, record &eping, analysis and reporting.
- Software engineers address quality activities @ applying technical metho6 and measures, condwting formal technical reviews, and performing well-planned software testing.
- SQA group is to assist the software team in achieving a high quality end product.

Role of an SQA group

1. Prepares an SQA plan for a project.

The plan is developed during project planning and is reviewed by all stakehDlders.

- **The plan identifi.es**
  - **Evaluations to be pErformEd**
  - **Audits and reviews to be performed**
  - **Standards that are applicable to the project**
  - **ProcedurES for error reporting and tracking**
  - **DDcur«ents to be produced by the SQA group**
  - **Amount of feB.dback provided to the softwa re project tear«**

Continue…

2. Participates in the development of the project's software process description.

3. Reviews software engineering activities ID verify compliance with the defined sDft\'vare process.

8. Audits designated software work products to verify compliance with those defined as part Df the software process.

S. The SQA group reviews selected wDrk products;

Software Reliability

- Software reliability is defined in statistical terms as "the probability of failure-fme Dperation Df a computer program in a specified environment for a specified time".
- What is meant by the term failure?
  - Failure is nonconformance tD sDftware requirements.
  - Correction of one failure may in fact result in the introduction of other errors that ultimately result in Dther failures.

Software eliability can be measured directed and estimated using historical and developmental data.

Measures of Reliability and Availability

- A simple measure of reliability is mean-time- between-failure (MTBF), where

$$MTBF = MTTF + MTTR$$

- The acronyms MTTF and MTTR are mean-time-to-failure and mean-time-to-repair, respectively.
- MTBF is a far more useful measure than defects/KLOC or defects/FP.
- Stated simply, an end-user is concerned with failures, not with the total error count.
- a reliability measure, we must develop a measure Df availability

Continue…

- **Software availability : It is the probability that a program is operating according to requirements at a given point in time and is defined as**

$$\text{Availability} = [MTTF/(MTTF + MTTR)] \times 100\%$$

- The MTBF reliability measure is equally sensitive to MTTF and MTTR.
- The availability measure is somewhat more sensitive to MTTR, an indirect measure of the maintainability of software

Quality Standards- ISO9000 And 9001

SQA Standards

- Software quality assurance standards can be classified into two main classes —

1. Software quality assurance managei?zent standards, including certification and assessment methodologies (quality management standards)
2. Software project development process standards (project process standards)

Quality Management Standards

- These focus on the organization's SQA system, infrastructure and requirements,
- while leaving the choice of methods and tools to the organization.
- With quality management standards, organizations can steadily assure that their software products achieve an acceptable level of quality.

Example —ISO 9000-3 and the Capability Maturity Model (CMM)

ISO 9001 Certification

- ISo (the International Organization for Standardization) is a worldwide federation of national standards bodies.
- ISO technical committees prepare the International Standards.
- ISO collaborates closely with the International Electro-technical Commission (IEC) on all matters of electro-technical standardization.
- International Standards are drafted in accordance with the rules given in the ISO/IEC Directives, Part 2.
- Draft of the International Standards adopted by the technical committees is circulated to the member bodies for voting.
- ISO 9001 was prepared by Technical Committee ISO/TC 176, Quality management and quality assurance, Subcommittee SC 2, Quality systems.

References

- Pressman, Roger S. Software engineering: a practitioner's approach. Palgrave Macmillan, 2005.
- Sommerville, Ian. Software Engineering: Pearson New International Edition. Pearson Education Limited, 2013.
- Jalote, Pankaj. An integrated approach to software engineering. Springer Science & Business Media, 2012.
- www.google.com

- # DIGITAL LEARNING CONTENT

0

Parul® University

www.paruluniversity.ac.in